

COMP713 Coursework 2 - Option B: P2P Messaging System

Student Name: Zhiyuan Zhang

Student ID: 22170060

1. Project Overview and Requirements

This project implements a small-scale peer-to-peer (P2P) messaging system (Option B). The system targets a decentralized network scenario where independent nodes need to communicate and collaborate without a central server. Key functional requirements include: running three independent peer nodes, sending/receiving messages via TCP sockets, utilizing Vector Clocks for event ordering, and implementing the Bully algorithm for leader election. The project aims to demonstrate fundamental distributed system concepts such as causal ordering, concurrency, and fault detection.

2. Architecture and Technology Stack

The system is built entirely using Python 3 and its standard library, employing a fully distributed P2P architecture. Each node runs two main threads: a TCP server thread (`accept_connections`) to listen for incoming JSON messages, and a heartbeat thread (`heartbeat_loop`) for fault detection. The main thread runs a command-line interface (CLI) for user interaction.

Communication Flow:

1. A node sends a JSON message to another node via TCP.
2. The receiving node's TCP server accepts the connection, parses the JSON, and processes the message.
3. For chat messages, the system merges vector clocks; for election messages, the Bully algorithm logic is triggered.

3. Implemented Features

- TCP Socket Communication (Completed and Tested): Nodes reliably exchange JSON messages using TCP sockets.
- Vector Clocks (Completed and Tested): Successfully implemented the causal ordering mechanism. Upon receiving a message, the local clock is merged with the sender's clock, and the receiver increments its own clock component .
- Bully Algorithm Leader Election (Completed and Tested):** Nodes correctly identify the node with the highest ID as the leader. The system handles concurrent election scenarios, and nodes correctly terminate their own election processes upon receiving a "LEADER announcement." Failure Detection and Timeout (Completed and Tested): If a node with a higher ID crashes, the remaining nodes wait for a 6-second timeout before declaring themselves the new Leader.
- Invalid Input Handling (Completed and Tested): The CLI correctly rejects unknown commands.

4. Communication and Distributed Systems Concepts

This project demonstrates several concepts from the COMP713 course. TCP sockets provide reliable, ordered byte streams. The message format uses ****JSON****. The Vector Clock mechanism is crucial for event ordering, allowing peers to determine causal relationships between messages. The Bully algorithm demonstrates coordination mechanisms in distributed systems. During testing, a classic "Split-Brain" race condition was observed; this was resolved by implementing strict timeout logic and ensuring nodes ignore election messages once they have confirmed themselves as the Leader. This highlights the trade-off between failure detection latency and system consistency.

5. Data and Message Design

- Messages are structured as JSON objects containing the following fields:
- type: CHAT, ELECTION, OK, LEADER, HEARTBEAT.
- sender: ID of the sending node.
- content: Chat content (used only for CHAT type).
- clock: Sender's current vector clock.
- candidate: ID of the node initiating the election.
- leader: ID of the newly elected Leader.

Each peer maintains the following state in memory: ``vector_clock`` (dictionary), ``leader_id`` (integer), ``in_election`` (boolean), and ``ok_received`` (set).

6. Testing and Verification

Testing was conducted using three PowerShell terminals.

Success: Executing ``chat Hello`` successfully updated the vector clocks across the peers.

Concurrent Election: Node 1 and Node 2 initiated elections simultaneously. Node 3 correctly selected itself as the Leader and broadcast a ``LEADER`` message. Upon receiving a duplicate request, Node 2 printed "I am already the Leader. Ignoring election." Fault Handling: Node 3 was terminated (via the ``quit`` command). Node 1 initiated an election. After a 6-second timeout, Node 2 became the new Leader, demonstrating the system's fault tolerance.

Invalid Input: Upon entering ``hello world``, the system displayed ``Unknown command...``.

7. Limitations and Future Improvements

Limitations: The system lacks persistent storage; if a node crashes, its state is lost. Fault detection relies on manually triggered elections rather than automatic ones. The node discovery mechanism is static, requiring hard-coded IP addresses and ports.

Future Improvements: A dynamic node discovery mechanism (e.g., using a broadcast protocol) could be implemented. Integrating a database (such as SQLite) for message persistence would enhance system robustness. Additionally, automatic election triggering based on heartbeat timeouts could be implemented.

8. Execution Instructions

Run ``python peer.py 1 5001 2:localhost:5002 3:localhost:5003``

Run ``python peer.py 2 5002 1:localhost:5001 3:localhost:5003``

Run ``python peer.py 3 5003 1:localhost:5001 2:localhost:5002``

Use CLI commands: `chat`, `election`, `status`, `lead`, `quit`